Intern Task Report

Task 7 — TTS Audio Caching System

Intern Name	Padigela Snehitha
Date of Task Given	20 May 2026
Supervisor	Vasudha Tayade
Captain	Aman Chauhan
Team Members	NA
Role in Team	Individual Task — Full Implementation

1. Executive Summary

The main objective of this task was to reduce repeated audio generation latency in a Text-to-Speech (TTS) system by implementing an efficient caching mechanism. The developed system stores generated audio outputs and reuses them whenever the same prompt is requested again. This implementation improved response time, reduced redundant processing, and optimized overall application performance. Additional handling for cache invalidation, prompt normalization, and repeated input management was also implemented to improve reliability and efficiency.

2. Task Overview

The assigned task focused on designing and implementing a caching system for the TTS optimization module. The module needed to handle repeated prompts efficiently and minimize unnecessary regeneration of audio outputs.

Objective:

• Reduce repeated audio generation latency
• Improve user response time
• Minimize unnecessary API or processing calls
• Improve overall system performance and scalability

Scope of Work:

• Prompt normalization
• Cache layer implementation
• Audio file storage and retrieval
• Cache hit and cache miss handling
• Cache invalidation logic
• Functional and performance testing

3. Technologies Used

Programming Language:

• Python

Libraries and Tools:

• os – file and directory management
• hashlib – generating unique cache keys
• json – metadata handling
• gTTS / TTS API – speech generation
• VS Code – development environment

Concepts Used:

• File-based caching
• Hash generation
• Audio optimization
• Cache invalidation

4. Implementation Details

The implementation was developed using a file-based caching strategy. Every input prompt was normalized and converted into a unique cache key using the hashlib library. The cache key was then used as the filename for storing audio files.

Workflow Followed:

1. Receive user prompt
2. Normalize the input text
3. Generate a unique hash key
4. Search the cache directory
5. Return cached audio if available
6. Generate new audio if cache is missing
7. Save generated audio to cache folder
8. Apply cache invalidation when required

5. Code Snippets

Prompt Normalization:

```
text = "  Hello World  "
normalized = " ".join(text.lower().split())
```

Cache Key Generation:

```
import hashlib

cache_key = hashlib.md5(normalized.encode()).hexdigest()
```

Cache Lookup and Storage:

```
if os.path.exists(cache_path):
    return cached_audio
else:
    tts.save(cache_path)
```

6. Testing & Validation

Different types of testing were performed to ensure proper functionality and reliability of the caching system.

Functional Testing:

• Verified cache creation
• Verified cache retrieval
• Checked proper audio storage

Edge Case Testing:

• Different capitalization inputs
• Extra spaces in prompts
• Repeated prompt requests
• Empty input handling

Performance Testing:

Response times were compared between first-time audio generation and cached audio retrieval. Cached outputs showed significantly faster response times.

7. Results & Outcome

The implemented caching system successfully reduced repeated audio generation latency and improved the overall efficiency of the application.

Achievements:

• Faster repeated prompt handling
• Reduced unnecessary processing
• Improved response speed
• Better resource utilization
• Successful cache invalidation support

8. Challenges & Solutions

Challenge 1: Duplicate cache entries caused by capitalization differences.
Solution: Implemented lowercase normalization.

Challenge 2: Different spacing formats created multiple cache files.
Solution: Added whitespace cleanup and formatting normalization.

Challenge 3: Cache storage continuously increased.
Solution: Implemented cache invalidation based on file age and cleanup logic.

9. Conclusion

The TTS Audio Caching System successfully optimized audio generation performance by introducing reusable audio caching functionality. The implementation reduced repeated processing and improved application responsiveness. This task provided practical experience in optimization techniques, caching strategies, file handling, and performance improvement.

10. References

• Python Documentation — https://docs.python.org/3/
• gTTS Documentation — https://gtts.readthedocs.io/
• Flask Documentation — https://flask.palletsprojects.com/
• Hashlib Documentation — https://docs.python.org/3/library/hashlib.html
• Visual Studio Code — https://code.visualstudio.com/